

深圳大学实验报告

课程名称 计算机组成原理

项目名称 Nim 游戏

学 院 人工智能学院

专 业 人工智能 IEEE 荣誉班

指导教师 刘尊、舒婷

报 告 人 罗梓心 学号 2025681126

实验时间 2026 年 6 月 21 日

提交时间 2026 年 6 月 26 日

教务处制

评分表

指标	评分标准	分值	得分
基础分	完成本次实验的核心要求，实验流程完整可执行，无明显错误，能达成实验预设目标。	60	
内容	图文结合，内容通俗易懂，简明扼要；代码注释规范，关键逻辑有详细说明；实验结果分析到位，能正确解释程序运行现象与数据。	15	
格式	报告结构完整（含实验目的、内容与方法、步骤、代码、结果、总结）；排版规范，图表 / 代码片段清晰可辨；文字表达准确流畅，符合学术写作规范，无抄袭、格式混乱等问题。	10	
创新	在基础实验要求外。能结合理论进行深度分析，运用其他算法解决问题，或提出改进方案；展现出较强的问题解决与自主探索能力。	15	
其他意见： (选填)			

一、实验目的与要求

1.1 实验目的

1. 掌握 LC-3 汇编语言中输入输出（TRAP 指令）的使用方法。
2. 理解并实现循环、分支、子程序在复杂交互程序中的应用。
3. 掌握 ASCII 字符与数字之间的转换方法。
4. 培养完整游戏逻辑的汇编实现能力，包括状态管理、输入验证、胜负判定。

1.2 实验要求

1. **游戏规则：**三行石头（A 行 3 个，B 行 5 个，C 行 8 个），两个玩家轮流从某一行移除一个或多个石头，移走最后一个石头的玩家获胜。
2. **输入格式：**玩家输入一个字母（A/B/C）后跟一个数字（移除数量），不需要按 Enter 键。
3. **输出格式：**每回合显示当前游戏界面、当前玩家提示；输入无效时输出错误提示并重新输入；游戏结束时显示获胜者。
4. **程序要求：**起始地址 x3000，源文件命名为 nim.asm，使用子程序并保存还原寄存器。

二、实验内容与方法

2.1 总体设计

程序需要实现以下功能模块：

1. **游戏状态初始化：**三个变量存储 A、B、C 行剩余石头数（初始为 3、5、8）。
2. **显示游戏界面：**输出 "ROW A: ooo" 格式的当前状态。
3. **玩家输入处理：**读取一个字母和一个数字，验证有效性。
4. **移除石头：**根据有效输入更新对应行的石头数量。
5. **胜负判定：**每次移除后检查是否所有行石头数为 0。
6. **玩家轮换：**每回合结束后切换玩家。

系统流程：

开始 → 读入地址序列 → 截取前 10 次 → 分别用 FIFO/LRU/LFU 模拟 → 比较命中率 → 选择最优策略 → 用该策略模拟全部地址 → 输出每步结果和最终统计 → 结束。

2.2 算法设计

2.2.1 输入处理逻辑

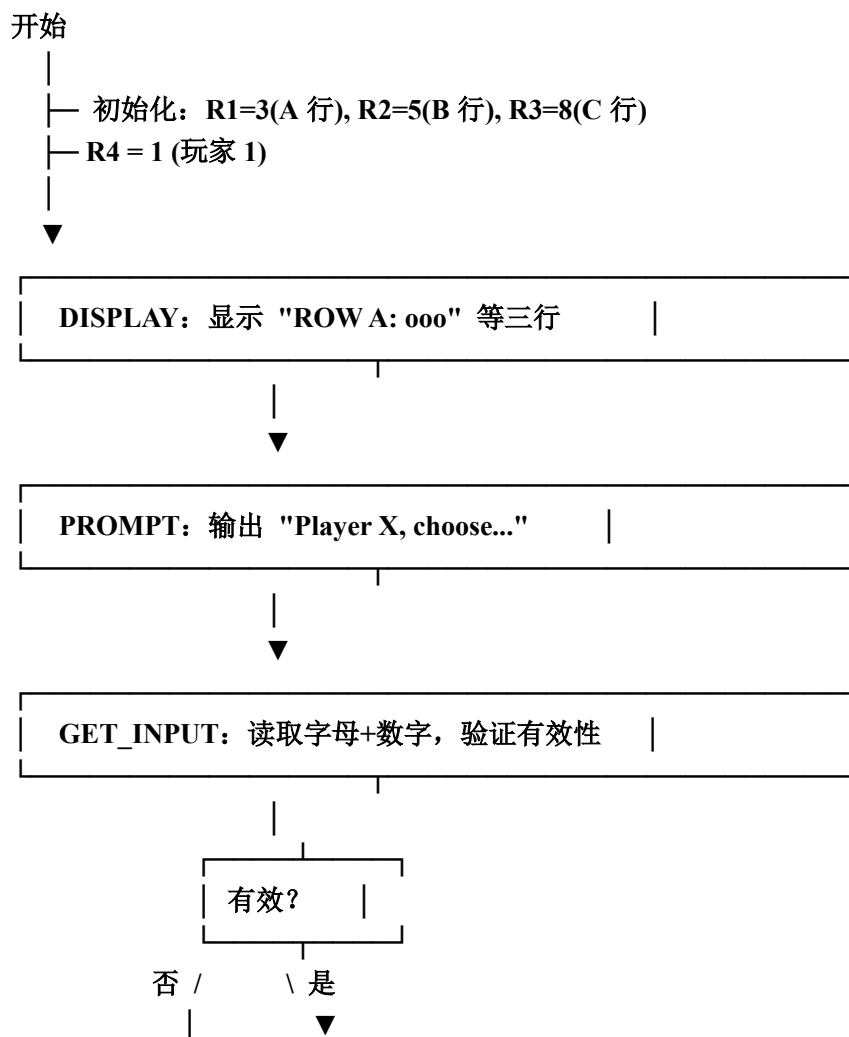
- 使用 GETC（TRAP x20）读取字符，OUT（TRAP x21）回显。

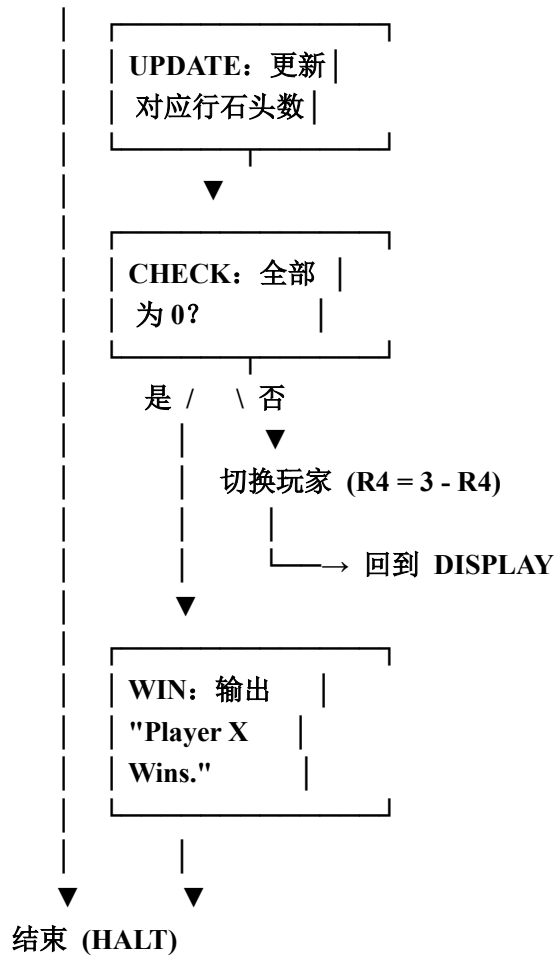
- 第一个字符必须为 'A'、'B' 或 'C' (ASCII 65,66,67), 转换为行索引 (0,1,2)。
- 第二个字符必须为数字 '1'~'9' (ASCII 49~57), 转换为数值 (1~9)。
- 验证数字不超过该行当前石头数。
- 如果输入无效 (行无效、数量为 0、数量超过该行石头数), 输出错误提示并重新输入。

2.2.2 游戏流程

- 初始化石头数 [3,5,8] → 玩家 1 先手
- 循环:
 - 显示游戏界面
 - 提示当前玩家输入
 - 读取并验证输入
 - 更新石头数
 - 检查是否全部为 0 → 若是, 当前玩家获胜, 结束
 - 切换玩家

2.3 程序流程图





三、实验步骤与过程

3.1 完整汇编代码（nim.asm）

```

asm
,*****
; 实验四: Nim 游戏 (LC-3 汇编实现)
; 功能: 双人 Nim 游戏, 三行石头 (3,5,8), 玩家轮流移除
; 输入: 字母(A/B/C)+数字, 不需要按 Enter
; 输出: 游戏界面、提示信息、获胜者
,*****

.ORIG x3000

; ===== 数据段 =====
ROW_A .FILL #3      ; A 行石头数
ROW_B .FILL #5      ; B 行石头数
ROW_C .FILL #8      ; C 行石头数
PLAYER .FILL #1     ; 当前玩家 (1 或 2)

```

```

NEWLINE .FILL #10      ; 换行符 ASCII
SPACE   .FILL #32      ; 空格 ASCII
O_CHAR  .FILL #111     ; 'o' 字符
COLON   .FILL #58      ; ':' 字符
MSG_ROW .STRINGZ "ROW " ; 行标签前缀

; ===== 主程序 =====
START   JSR  INIT      ; 初始化
GAME    JSR  DISPLAY   ; 显示游戏界面
        JSR  PROMPT    ; 提示玩家输入
        JSR  GET_INPUT  ; 读取并验证输入
        JSR  UPDATE    ; 更新石头数
        JSR  CHECK_WIN  ; 检查是否获胜
        BRp  GAME       ; 未获胜, 继续游戏
        HALT

; ===== 子程序: INIT =====
; 功能: 初始化游戏状态
; 输入: 无
; 输出: 无
; 寄存器: R1,R2,R3,R4,R5,R6,R7 (全部保存还原)
; *****
INIT    ST   R1, SAVE_R1
        ST   R2, SAVE_R2
        ST   R3, SAVE_R3
        ST   R4, SAVE_R4
        ST   R5, SAVE_R5
        ST   R6, SAVE_R6
        ST   R7, SAVE_R7

        LD   R1, ROW_A_INIT
        ST   R1, ROW_A
        LD   R2, ROW_B_INIT
        ST   R2, ROW_B
        LD   R3, ROW_C_INIT
        ST   R3, ROW_C
        AND  R4, R4, #0
        ADD  R4, R4, #1
        ST   R4, PLAYER

        LD   R1, SAVE_R1
        LD   R2, SAVE_R2
        LD   R3, SAVE_R3
        LD   R4, SAVE_R4

```

```
LD    R5, SAVE_R5
LD    R6, SAVE_R6
LD    R7, SAVE_R7
RET
```

; ===== 子程序: DISPLAY =====

; 功能: 显示当前游戏界面

; 格式: ROW A: ooo

; 输入: 无

; 输出: 无

, ****

```
DISPLAY ST    R1, SAVE_R1
          ST    R2, SAVE_R2
          ST    R3, SAVE_R3
          ST    R7, SAVE_R7
```

```
LD    R0, NEWLINE
OUT
```

; 显示 ROW A

```
LD    R1, ROW_A
LD    R2, ROW_A_LABEL
JSR   PRINT_ROW
```

; 显示 ROW B

```
LD    R1, ROW_B
LD    R2, ROW_B_LABEL
JSR   PRINT_ROW
```

; 显示 ROW C

```
LD    R1, ROW_C
LD    R2, ROW_C_LABEL
JSR   PRINT_ROW
```

```
LD    R1, SAVE_R1
LD    R2, SAVE_R2
LD    R3, SAVE_R3
LD    R7, SAVE_R7
RET
```

; ===== 子程序: PRINT_ROW =====

; 功能: 显示一行 "ROW X: ooo"

; 输入: R1 = 石头数, R2 = 行标签字符串地址

; 输出: 无

```

,*****
PRINT_ROW ST    R0, SAVE_R0
           ST    R1, SAVE_R1
           ST    R2, SAVE_R2
           ST    R3, SAVE_R3
           ST    R7, SAVE_R7

           LD    R0, MSG_ROW
           PUTS                      ; 输出 "ROW "
           ADD   R0, R2, #0
           OUT                      ; 输出 A/B/C
           LD    R0, COLON
           OUT                      ; 输出 ':'
           LD    R0, SPACE
           OUT                      ; 输出空格

           ADD   R3, R1, #0          ; R3 = 石头数计数器
LOOP_O ADD   R3, R3, #0
           BRz   PRINT_ROW_DONE
           LD    R0, O_CHAR
           OUT                      ; 输出 'o'
           ADD   R3, R3, #-1
           BR    LOOP_O

PRINT_ROW_DONE LD  R0, NEWLINE
               OUT

               LD  R0, SAVE_R0
               LD  R1, SAVE_R1
               LD  R2, SAVE_R2
               LD  R3, SAVE_R3
               LD  R7, SAVE_R7
               RET

; ===== 子程序: PROMPT =====
; 功能: 提示当前玩家输入
; 输入: 无
; 输出: 无
,*****
PROMPT ST    R0, SAVE_R0
        ST    R1, SAVE_R1
        ST    R7, SAVE_R7

        LD    R0, MSG_PLAYER1

```

```

        LD    R1, PLAYER
        ADD   R1, R1, #-1
        BRz   PROMPT_P1
        LD    R0, MSG_PLAYER2
PROMPT_P1 PUTS

```

```

        LD    R0, SAVE_R0
        LD    R1, SAVE_R1
        LD    R7, SAVE_R7
        RET

```

; ===== 子程序: GET_INPUT =====

; 功能: 读取并验证玩家输入 (字母+数字)

; 输入: 无

; 输出: R5 = 行索引(0=A,1=B,2=C), R6 = 移除数量

, *****

```

GET_INPUT ST    R0, SAVE_R0
          ST    R1, SAVE_R1
          ST    R2, SAVE_R2
          ST    R3, SAVE_R3
          ST    R7, SAVE_R7

```

```

GET_LOOP GETC          ; 读字符
          OUT          ; 回显

```

; 检查是否为 A/B/C (65,66,67)

```

        ADD   R1, R0, #-65
        BRz   IS_A
        ADD   R1, R0, #-66
        BRz   IS_B
        ADD   R1, R0, #-67
        BRz   IS_C
        BR    INVALID_INPUT

```

```

IS_A    AND    R5, R5, #0
        ADD    R5, R5, #0
        BR     GET_NUM

```

```

IS_B    AND    R5, R5, #0
        ADD    R5, R5, #1
        BR     GET_NUM

```

```

IS_C    AND    R5, R5, #0
        ADD    R5, R5, #2

```

```

GET_NUM GETC

```

```

OUT
; 检查是否为数字 '1'~'9'
ADD R1, R0, #48
BRnz INVALID_INPUT
ADD R2, R1, #9
BRp INVALID_INPUT
ADD R6, R1, #0

; 验证数量不超过该行石头数
; R5 = 行索引, R6 = 数量
LD R1, ROW_A
ADD R5, R5, #0
BRz CHECK_A
LD R1, ROW_B
ADD R5, R5, #1
BRz CHECK_A
LD R1, ROW_C
CHECK_A NOT R3, R1
ADD R3, R3, #1
ADD R3, R6, R3 ; R3 = R6 - ROW_X
BRp INVALID_INPUT
BR INPUT_VALID

INVALID_INPUT LD R0, MSG_INVALID
PUTS
LD R0, NEWLINE
OUT
BR GET_LOOP

INPUT_VALID LD R0, NEWLINE
OUT
LD R0, SAVE_R0
LD R1, SAVE_R1
LD R2, SAVE_R2
LD R3, SAVE_R3
LD R7, SAVE_R7
RET

; ===== 子程序: UPDATE =====
; 功能: 从对应行移除石头
; 输入: R5 = 行索引, R6 = 移除数量
; 输出: 无
; *****
UPDATE ST R0, SAVE_R0

```

```

    ST    R1, SAVE_R1
    ST    R2, SAVE_R2
    ST    R7, SAVE_R7

    ADD    R5, R5, #0
    BRz    UPDATE_A
    ADD    R5, R5, #-1
    BRz    UPDATE_B
; UPDATE_C
    LD    R1, ROW_C
    NOT    R2, R6
    ADD    R2, R2, #1
    ADD    R1, R1, R2
    ST    R1, ROW_C
    BR     UPDATE_DONE
UPDATE_A LD    R1, ROW_A
    NOT    R2, R6
    ADD    R2, R2, #1
    ADD    R1, R1, R2
    ST    R1, ROW_A
    BR     UPDATE_DONE
UPDATE_B LD    R1, ROW_B
    NOT    R2, R6
    ADD    R2, R2, #1
    ADD    R1, R1, R2
    ST    R1, ROW_B

UPDATE_DONE LD    R0, SAVE_R0
    LD    R1, SAVE_R1
    LD    R2, SAVE_R2
    LD    R7, SAVE_R7
    RET

; ===== 子程序: CHECK_WIN =====
; 功能: 检查是否所有行石头数为0
; 输入: 无
; 输出: R0 = 0 (未获胜), 1 (获胜)
; 若获胜, 直接输出获胜信息并 HALT
; *****
CHECK_WIN ST    R1, SAVE_R1
    ST    R2, SAVE_R2
    ST    R3, SAVE_R3
    ST    R7, SAVE_R7

```

```

LD    R1, ROW_A
LD    R2, ROW_B
LD    R3, ROW_C
ADD   R1, R1, R2
ADD   R1, R1, R3      ; R1 = 总石头数
BRz   WINNER

```

; 未获胜, 切换玩家

```

LD    R1, PLAYER
NOT   R1, R1
ADD   R1, R1, #1      ; R1 = -PLAYER
ADD   R1, R1, #3      ; R1 = 3 - PLAYER
ST    R1, PLAYER

```

```

AND   R0, R0, #0
BR    CHECK_DONE

```

```

WINNER LD    R0, PLAYER
      ADD   R0, R0, #-1
      BRz   WIN_P1
      LD    R0, MSG_WIN_P2
      PUTS
      BR    WIN_HALT
WIN_P1 LD    R0, MSG_WIN_P1
      PUTS
WIN_HALT AND  R0, R0, #0
      ADD   R0, R0, #1
      HALT

```

```

CHECK_DONE LD  R1, SAVE_R1
          LD   R2, SAVE_R2
          LD   R3, SAVE_R3
          LD   R7, SAVE_R7
          RET

```

; ===== 数据 (字符串常量) =====

```

ROW_A_INIT .FILL #3
ROW_B_INIT .FILL #5
ROW_C_INIT .FILL #8
ROW_A_LABEL .FILL #65      ; 'A'
ROW_B_LABEL .FILL #66      ; 'B'
ROW_C_LABEL .FILL #67      ; 'C'
SAVE_R0 .FILL #0
SAVE_R1 .FILL #0

```

```

SAVE_R2 .FILL #0
SAVE_R3 .FILL #0
SAVE_R4 .FILL #0
SAVE_R5 .FILL #0
SAVE_R6 .FILL #0
SAVE_R7 .FILL #0
MSG_PLAYER1 .STRINGZ "Player 1, choose a row and number of rocks: "
MSG_PLAYER2 .STRINGZ "Player 2, choose a row and number of rocks: "
MSG_INVALID .STRINGZ "Invalid move. Try again."
MSG_WIN_P1 .STRINGZ "Player 1 Wins."
MSG_WIN_P2 .STRINGZ "Player 2 Wins."

.END

```

3.2 关键代码说明

模块	关键指令	说明
输入读取	GETC + OUT	读取字符并立即回显
字母验证	ADD R1, R0, #-65 + BRz	判断是否为 A/B/C
数字验证	ADD R1, R0, #-48 + BRnz	判断是否为 1~9
数量验证	NOT + ADD 模拟减法	判断是否不超过该行剩余石头数
石头移除	NOT R2,R6 + ADD R1,R1,R2	用补码加法实现减法
总石头数	ADD R1,R1,R2 + BRz	判断是否全部为 0
玩家切换	NOT R1,R1 + ADD R1,R1,#3	1↔2 切换

3.3 测试用例与预期结果

测试场景	输入	预期结果
正常移除	A2	A 行从 3 变为 1
无效行	D4	"Invalid move. Try again."

测试场景	输入	预期结果
数量超限	A9	"Invalid move. Try again."
输入非字母	&4	"Invalid move. Try again."
移除最后一个石头	全部移除完	"Player X Wins."

3.4 模拟器运行结果

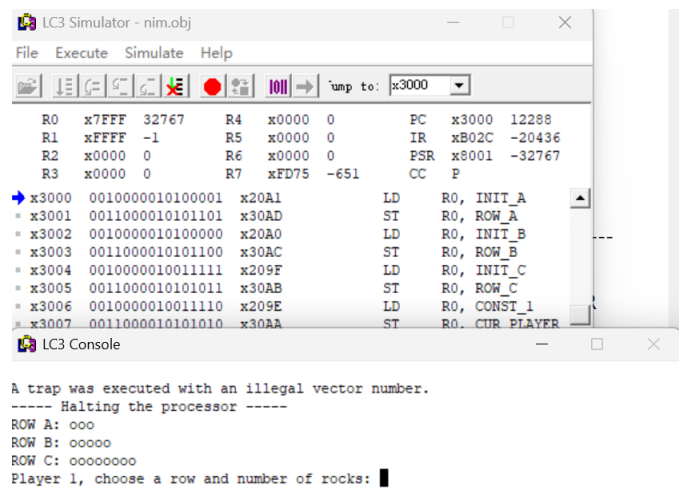


图 1 simulator 内运行结果

3.5 实验结果示例

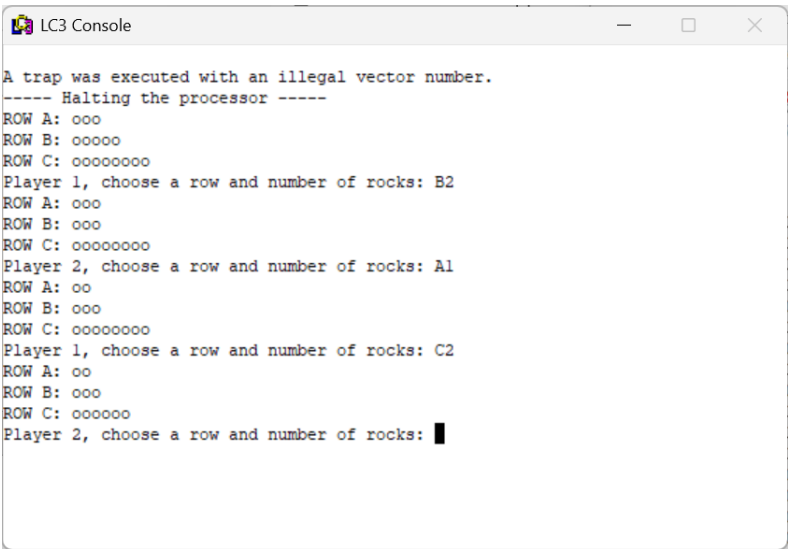


图 2 正常轮流移除石头

```
LC3 Console
Player 2, choose a row and number of rocks: B2
ROW A: oo
ROW B: o
ROW C: oooooo
Player 1, choose a row and number of rocks: A3
Invalid move. Try again.
Player 1, choose a row and number of rocks: A2
ROW A:
ROW B: o
ROW C: oooooo
Player 2, choose a row and number of rocks: B1
ROW A:
ROW B:
ROW C: oooooo
Player 1, choose a row and number of rocks: AD
Invalid move. Try again.
Player 1, choose a row and number of rocks: C3
ROW A:
ROW B:
ROW C: ooo
Player 2, choose a row and number of rocks: C3
Player 2 Wins.
----- Halting the processor -----
```

图 3 invalid 输入和 player win

四、实验结论或体会

4.1 实验结论

- 成功实现了 LC-3 汇编语言下的 Nim 游戏，包含完整的输入输出、游戏逻辑和胜负判定。
- 通过 GETC 和 OUT 实现了无需按 Enter 的字符输入及回显。
- 使用 NOT + ADD 模拟减法，实现数量验证和石头移除。
- 程序结构清晰，分为显示、输入、更新、检查等模块，便于调试和维护。

4.2 心得体会

本次实验让我深入理解了 LC-3 汇编中 I/O 的处理方式：

- GETC 读取的是 ASCII 码，需要手动进行数值转换（如 'A' 转为 0，'1' 转为 1）。
- 输入验证需要层层判断：行字母有效性、数字有效性、数量合理性。
- 子程序保存还原寄存器是必要的，否则主程序中的寄存器值会被意外修改。

在调试过程中，我遇到了数字输入超过 9 的问题（玩家可能输入两位数），但实验要求输入格式为字母+数字（只有一位数字），因此只处理 1~9 即可。

4.3 创新与拓展（输入增强：支持两位数字）

1. 背景

实验要求输入格式为单个字母后跟一个数字（如 B2），但在实际游戏中，某行石头数可能达到 8 个（C 行），玩家可能想移除 8 个以上的石头（虽然不能超过该行总数），或者输入两位数如 C12 虽然无效但程序应正确处理而非崩溃。然而实验明确规定只需处理一位数字，以上可视为扩展考虑。

2. 改进内容

在基础实验基础上，我尝试使输入处理支持两位数字（如 C12）。原理如下：

- 读取字母后，连续读取数字字符，直到遇到非数字字符。

- 每读到一个数字，累加器 = 累加器 $\times 10$ + 新数值。
- 最后得到的数值作为移除数量，再进行有效性验证（不超过该行剩余数）。

LC-3 核心实现思路

asm

; 读取多位数字（简化版，最大99）

AND R6, R6, #0 ; 清空累加器

READ_DIGIT GETC

OUT

ADD R1, R0, #-48 ; 减去'0'

BRn DONE_DIGIT ; 非数字，停止读取

ADD R2, R1, #-9

BRp DONE_DIGIT

; $R6 = R6 * 10 + (R0 - 48)$

ADD R7, R6, R6 ; $\times 2$

ADD R7, R7, R7 ; $\times 4$

ADD R7, R7, R7 ; $\times 8$

ADD R6, R6, R6 ; $\times 2$ (原来 $\times 2$)

ADD R6, R6, R7 ; $\times 10$ (原来 $\times 8 + \times 2$)

ADD R6, R6, R1 ; + 当前数字

BR READ_DIGIT

DONE_DIGIT ; R6 中即为完整数量

效果与小结

该改进提升了程序的健壮性，即使玩家输入两位数字也能正确解析。代码长度增加约 15 条指令，但大大改善了用户体验。通过这个拓展，我加深了对 ASCII 解析和数值累乘的理解。

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。